intrevw questions on collection java

Here's a **comprehensive list of Java Collection Framework interview questions —** grouped by difficulty level (Basic → Advanced), along with **short and clear answers** where useful.

🟢 Basic Level (Core Concepts)

1. **What is the Collection Framework in Java?**
   A unified architecture for storing and manipulating groups of objects. It includes interfaces (List, Set, Queue, Map) and their implementations (ArrayList, HashSet, HashMap, etc.).
2. **What are the main interfaces in the Java Collections Framework?**
   - **Collection** (root)
     - **List** → ArrayList, LinkedList, Vector
     - **Set** → HashSet, LinkedHashSet, TreeSet
     - **Queue** → PriorityQueue, LinkedList
   - **Map** → HashMap, TreeMap, LinkedHashMap, Hashtable
3. **What is the difference between Collection and Collections?**
   - **Collection** → Interface
   - **Collections** → Utility class (with static methods like `sort()`, `reverse()`, `synchronizedList()`).
4. **What is the difference between List, Set, and Map?**

Interface	Duplicates	Order	Example
List	✅ Yes	✅ Maintains	ArrayList
Set	❌ No	❌ No guarantee (except LinkedHashSet)	HashSet
Map	Keys ❌, Values ✅	✅ (LinkedHashMap)	HashMap

5. **What is the difference between Array and ArrayList?**
   - Array: Fixed size, can hold primitives.
   - ArrayList: Dynamic size, only objects, part of Collections.

6. **How does HashSet work internally?**
   Uses a **HashMap** internally. Each element is stored as a key in the map with a constant dummy value.
7. **How does HashMap work internally?**
   Based on **hashing** — it uses **hashCode()** and **equals()** methods. Collisions are handled using **linked lists** (or **balanced trees** since Java 8).
8. **Difference between HashMap and Hashtable?**

HashMap	Hashtable
Non-synchronized	Synchronized
Allows null keys/values	Doesn't allow null
Faster	Slower

9. **What is the difference between Iterator and ListIterator?**
   - **Iterator**: Can traverse only forward.
   - **ListIterator**: Can traverse both directions and modify the list.
10. **Can we use null as a key in HashMap?**
    Yes, only one `null` key is allowed (since keys must be unique).

🟡 Intermediate Level

11. **What is the difference between Comparable and Comparator?**
- **Comparable**: Natural ordering (implements `compareTo()`).
- **Comparator**: Custom ordering (implements `compare()`).

12. **What is the difference between HashSet and TreeSet?**
- HashSet: Unordered, faster.
- TreeSet: Sorted, slower (uses Red-Black Tree).

13. **What is the difference between ArrayList and LinkedList?**

ArrayList	LinkedList
Backed by dynamic array	Backed by doubly linked list
Fast random access	Fast insertion/deletion
Slower for add/remove in middle	Slower for get(index)

14. **What is the load factor in HashMap?**
    Default is **0.75**. When the map is 75% full, capacity is doubled to maintain efficiency.
15. **What happens when two keys have the same hash code in a HashMap?**
    A collision occurs — elements are stored in the same bucket using a linked list (or tree from Java 8 onward).
16. **What is the difference between fail-fast and fail-safe iterators?**
    - **Fail-fast**: Throws `ConcurrentModificationException` (e.g., ArrayList, HashMap).
    - **Fail-safe**: Doesn't throw exception; works on a clone (e.g., ConcurrentHashMap, CopyOnWriteArrayList).
17. **How does ConcurrentHashMap work internally?**
    Uses **segments and locks per bucket** (in Java 7) or **CAS + synchronized blocks** (in Java 8). It allows concurrent read/write.
18. **Why are HashMap keys immutable?**
    To ensure consistent `hashCode()` and `equals()` behavior. Mutable keys can cause incorrect lookups.
19. **What are differences between Collection and Stream API?**
    - Collection: Stores data.
    - Stream: Performs operations (filter, map, reduce) on data.
20. **What is LinkedHashMap used for?**
    Maintains insertion order or access order (used in LRU cache).

🔴 Advanced / Tricky Questions

21. **What is the time complexity of major operations in HashMap?**
    - Average: O(1) for put/get/remove
    - Worst: O(log n) (after treeification in collisions)
22. **What is treeification in HashMap (Java 8)?**
    When a bucket has more than 8 elements, the linked list is converted into a **balanced tree** to improve performance.
23. **Why are wrapper classes (like Integer) used in collections instead of primitives?**
    Collections store **objects**, not primitives.
24. **Can we synchronize a collection manually?**
    Yes, using:

java Copy code

```
List list = Collections.synchronizedList(new ArrayList<>());
```

25. **Difference between Enumeration, Iterator, and ListIterator?**
    - **Enumeration**: Legacy, only read.
    - **Iterator**: Read + remove.
    - **ListIterator**: Bidirectional + modify.
26. **How do you remove duplicates from a List in Java?**

java Copy code

```
List<Integer> list = Arrays.asList(1,2,3,2,1);
List<Integer> result = new ArrayList<>(new HashSet<>(list));
```

27. **When to use LinkedHashSet over HashSet?**
    When insertion order must be maintained.
28. **Difference between Poll() and Remove() in Queue?**
    - `poll()` returns `null` if queue is empty.
    - `remove()` throws `NoSuchElementException`.
29. **What is PriorityQueue and how does it work?**
    Queue that orders elements based on **natural order** or **Comparator**; implemented as a **heap**.
30. **Why is TreeMap slower than HashMap?**
    TreeMap is backed by a **Red-Black Tree**, providing **O(log n)** performance for operations, while HashMap gives **O(1)** average time.

Would you like me to give you a **focused list of frequently asked questions for interviews at product-based companies (like Amazon, JP Morgan, Barclays, etc.) —** with *sample code snippets* for answers?

yes

Perfect 👍

Here's a **focused list of the most frequently asked Java Collections questions —** especially useful for **product-based company interviews** (Amazon, JP Morgan, Barclays, etc.) — along with **short explanations + code snippets** to help you answer confidently.

🧠 1. How does HashMap work internally?

✅ **Most asked question**

Answer:

HashMap uses a **hashing mechanism** to store key-value pairs.

Each key's `hashCode()` determines the **bucket index**.

If multiple keys map to the same bucket (collision), entries are stored in a **linked list** (or **balanced tree** from Java 8).

java Copy code

```java
Map<String, Integer> map = new HashMap<>();
map.put("A", 1);
map.put("B", 2);
```

Internally:

1. Compute hash of "A" → find bucket index.
2. Store Entry(`key` , `value` , `hash` , `next`).
3. If collision, use equals() to check and handle.

Time Complexity:

- Average: O(1)
- Worst: O(log n) (after treeification in collisions)

⚙️ 2. What is the difference between HashMap and ConcurrentHashMap?

Feature	HashMap	ConcurrentHashMap
Thread-safe	❌ No	✅ Yes

Feature	HashMap	ConcurrentHashMap
Null keys	✅ Allowed (1)	❌ Not allowed
Iterator	Fail-fast	Fail-safe
Locking	Entire map	Segment / bucket level locking (Java 7), CAS + synchronized (Java 8)

Example:

java Copy code

```java
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();
map.put("A", 1);
map.put("B", 2);
```

🧩 3. Difference between Comparable and Comparator

✅ Used to sort custom objects.

java Copy code

```java
class Student implements Comparable<Student> {
    int id; String name;
    public int compareTo(Student s) {
        return this.id - s.id; // sort by id
    }
}

Comparator<Student> nameComparator = (s1, s2) -> s1.name.compareTo(s2.name);
```

Interface	Package	Method	Used for
Comparable	java.lang	compareTo()	Natural ordering
Comparator	java.util	compare()	Custom ordering

🧮 4. What is the difference between ArrayList and LinkedList?

Feature	ArrayList	LinkedList
Structure	Dynamic array	Doubly linked list
Access	O(1)	O(n)
Insert/Delete (middle)	O(n)	O(1)
Best for	Random access	Frequent insert/delete

Example:

java Copy code

```java
List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
```

⚡ 5. What is fail-fast vs fail-safe iterator?

✅ Frequently asked in concurrency topics.

- **Fail-fast:** Throws `ConcurrentModificationException` if structure changes during iteration. (e.g., `ArrayList`, `HashMap`)
- **Fail-safe:** Works on a clone — no exception. (e.g., `ConcurrentHashMap`, `CopyOnWriteArrayList`)

java Copy code

```java
List<Integer> list = new ArrayList<>(List.of(1,2,3));
for (Integer i : list) {
    list.add(4); // Throws ConcurrentModificationException
}
```

🔒 6. How to make a Collection thread-safe?

Using Collections utility class:

java Copy code

```
List<Integer> syncList = Collections.synchronizedList(new ArrayList<>());
Map<String, Integer> syncMap = Collections.synchronizedMap(new HashMap<>());
```

Or use concurrent classes:

- `ConcurrentHashMap`
- `CopyOnWriteArrayList`
- `BlockingQueue`

🧱 7. What is the difference between HashSet and TreeSet?

Feature	HashSet	TreeSet
Ordering	Unordered	Sorted (Natural/Comparator)
Backed by	HashMap	TreeMap
Null elements	1 null allowed	No nulls

Example:

java Copy code

```
Set<Integer> set = new TreeSet<>();
set.add(5);
set.add(1);
set.add(3);
System.out.println(set); // [1, 3, 5]
```

👛 8. How does LinkedHashMap maintain order?

It maintains a **doubly linked list** of entries — preserving insertion order (or access order if configured).

Used in:

👉 LRU Cache implementation.

Example (LRU cache style):

java Copy code

```java
Map<Integer, String> lru = new LinkedHashMap<>(3, 0.75f, true) {
    protected boolean removeEldestEntry(Map.Entry<Integer, String> eldest) {
        return size() > 3;
    }
};
```

---

9. What is PriorityQueue in Java?

Queue that orders elements by **natural order** or **Comparator** (implemented using a heap).

Example:

java Copy code

```java
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(10);
pq.add(5);
pq.add(20);
System.out.println(pq.poll()); // 5 (min-heap)
```

---

10. Difference between HashMap and LinkedHashMap

Feature	HashMap	LinkedHashMap
Order	Unordered	Maintains insertion/access order
Performance	Slightly faster	Slightly slower
Use case	General-purpose map	Caching (LRU, MRU)

---

11. How to remove duplicates from a List?

✅ Common coding question.

Using Set:

java Copy code

```java
List<Integer> list = Arrays.asList(1,2,3,2,1);
List<Integer> result = new ArrayList<>(new LinkedHashSet<>(list));
System.out.println(result); // [1,2,3]
```

🧩 12. What is the difference between HashMap and TreeMap?

Feature	HashMap	TreeMap
Ordering	Unordered	Sorted (by key)
Null keys	Allowed	Not allowed
Performance	O(1)	O(log n)

🧠 13. Why are HashMap keys immutable?

If keys change after insertion, their hash value may change — making them unreachable in the map.

🔍 14. Internal working of HashSet

HashSet uses a **HashMap internally**.
Each element is stored as a **key**, with a constant dummy value (`PRESENT`).

java Copy code

```java
private static final Object PRESENT = new Object();
```

⚖️ 15. How to sort a Map by values?

Using Stream API:

java Copy code

```java
Map<String, Integer> map = Map.of("A", 3, "B", 1, "C", 2);
map.entrySet().stream()
    .sorted(Map.Entry.comparingByValue())
    .forEach(System.out::println);
```

🧩 Bonus: Tricky Questions

- What happens if hashCode() always returns same value?
  → All entries go in same bucket → O(n) performance.
- Can two unequal objects have same hashCode()?
  → Yes (collision), but equals() differentiates them.
- Why does TreeSet/TreeMap need Comparable or Comparator?
  → For sorting order definition.
- Why is load factor important?
  → Controls when rehashing occurs (default = 0.75).

Would you like me to now give you a "**Top 15 Collection-based Coding Problems**" (with examples like sorting map by value, removing duplicates, grouping, merging maps